

```
body { font-family: Helvetica, sans-serif; font-size: 10pt; line-height: 1.5; color: #333333; } h1 { font-size: 20pt; font-weight: bold; color: #002D62; margin-top: 20px; margin-bottom: 10px; } h2 { font-size: 14pt; font-weight: bold; color: #4682B4; margin-top: 15px; margin-bottom: 8px; border-bottom: 1px solid #ddd; } h3 { font-size: 11pt; font-weight: bold; color: #333333; margin-top: 12px; margin-bottom: 6px; } p { margin-bottom: 10px; } table { border-collapse: collapse; width: 100%; margin-bottom: 15px; } th { background-color: #f2f2f2; font-weight: bold; text-align: left; padding: 6px; border: 1px solid #dddddd; } td { padding: 6px; border: 1px solid #dddddd; } ul { margin-bottom: 10px; padding-left: 20px; } li { margin-bottom: 4px; }
```

Buds Connect - Technical Evidence Document

Response to App Store Review Guideline 5.2.1

App Name: Buds Connect
Bundle ID: tech.miguellabs.GalaxyBudsMac
Developer: Felipe Miguel (MiguelLabs)
Submission ID: aa266298-c95f-4d42-b504-852e540031d9
Date: July 15, 2026
License: GNU General Public License v3.0 (GPLv3)
Source Code: <https://github.com/felipesilvax1/GalaxyBudsMacintosh>

Table of Contents

1. [Executive Summary](#1-executive-summary)
2. [Technical Architecture](#2-technical-architecture)
3. [Communication Protocol Analysis](#3-communication-protocol-analysis)
4. [Dependency Audit - Zero Samsung Code](#4-dependency-audit-zero-samsung-code)
5. [Permissions & Sandboxing](#5-permissions--sandboxing)
6. [App Store Precedents](#6-app-store-precedents)
7. [Legal Framework for Interoperability](#7-legal-framework-for-interoperability)
8. [Bluetooth SIG Standards Compliance](#8-bluetooth-sig-standards-compliance)
9. [Privacy & Data Collection](#9-privacy--data-collection)
10. [Conclusion](#10-conclusion)

1. Executive Summary

Buds Connect is a lightweight, open-source macOS menu bar utility that allows users to monitor battery status and adjust noise control settings on compatible Bluetooth earbuds they already own.

Key facts:

- [Yes] Uses **exclusively Apple's first-party IOBluetooth framework** - no third-party SDKs
- [Yes] Communicates via **RFCOMM/SPP** - an open, standardized Bluetooth transport protocol
- [Yes] Contains **zero Samsung code, SDKs, libraries, assets, or trademarks**
- [Yes] Has **zero third-party dependencies** - only Apple system frameworks
- [Yes] Makes **zero network connections** - fully offline, no telemetry sent externally
- [Yes] Is **fully sandboxed** with only Bluetooth entitlement
- [Yes] Source code is **100% publicly auditable** under GPLv3
- [Yes] Protocol was reverse-engineered from **publicly observable Bluetooth traffic** - a legally protected practice

2. Technical Architecture

2.1 System Frameworks Used

The app imports **exclusively Apple first-party frameworks**. Below is the complete list of every framework imported across all Swift source files in the project:

Framework	Provider	Purpose
Foundation	Apple	Core data types and utilities
IOBluetooth	Apple	Classic Bluetooth (RFCOMM/SPP) communication
Observation	Apple	@Observable macro for reactive state
SwiftUI	Apple	User interface framework
Cocoa	Apple	NSPanel, NSWindow for connection popup
WidgetKit	Apple	Desktop widget support
MetricKit	Apple	Local-only diagnostic telemetry
AppIntents	Apple	Siri Shortcuts integration

There are no third-party frameworks, SDKs, or libraries in the project.

2.2 Communication Stack Diagram

```

+-----+
| Buds Connect (Swift App) |
| - 100% original Swift code - |
+-----+
| Apple IOBluetooth API   |
| (public, documented macOS framework) |
+-----+
| macOS IOBluetooth Framework |
| - Apple's Bluetooth Classic stack - |
+-----+
| RFCOMM / SPP           |
| (open Bluetooth SIG standard) |
+-----+
| Bluetooth Radio (Host Hardware) |
| - Mac's built-in Bluetooth controller - |
+-----+
| Bluetooth Classic RF    |
| (2.4 GHz ISM band)     |
+-----+
| Compatible Bluetooth Earbuds |
| - User's own paired hardware - |
+-----+

```

At no layer does any Samsung proprietary code, SDK, or API exist.

2.3 RFCOMM Connection Sequence (Source Code Evidence)

File:
[BluetoothManager.swift](https://github.com/felipesilvax1/GalaxyBudsMacintosh/blob/master/GalaxyBudsMac/GalaxyBudsMac/Core/Bluetooth/BluetoothManager.swift)

The connection is established using **only standard Apple IOBluetooth API calls**:

```

// Step 1: Open base Bluetooth connection (IOBluetooth API)
device.openConnection()

// Step 2: Query the device's service catalog via SDP (standard protocol)
device.performSDPQuery(nil)

// Step 3: Find SPP service record by UUID
let record = device.getServiceRecord(for: uuid)

// Step 4: Get RFCOMM channel ID from the service record
serviceRecord.getRFCOMMChannelID(&channelID)

// Step 5: Open RFCOMM channel (standard Bluetooth transport)
device.openRFCOMMChannelSync(&tempChannel, withChannelID: channelID, delegate: self)

```

Every single call above is a **public, documented Apple API**. The full source code is available for verification at the GitHub repository.

3. Communication Protocol Analysis

3.1 Transport Layer: RFCOMM/SPP (Open Standard)

The app communicates using **Serial Port Profile (SPP)** over **RFCOMM** - both are open standards defined by the Bluetooth Special Interest Group (Bluetooth SIG). SPP emulates a serial port connection over Bluetooth.

- **RFCOMM** is defined in the Bluetooth Core Specification (publicly available at [bluetooth.com](https://www.bluetooth.com/specifications/assigned-numbers/rfcomm))
- **SPP** is a Bluetooth Profile built on RFCOMM (publicly documented by Bluetooth SIG)
- **No licensing is required** for software applications that use the OS's existing Bluetooth stack

3.2 Application Layer: Message Format

On top of the standard RFCOMM transport, the app sends and receives structured binary messages. The message format was documented through **independent reverse engineering of publicly observable Bluetooth traffic**:

```

+-----+-----+-----+-----+-----+-----+
| SOM | Header | Msg ID | Payload | CRC16 | EOM |
| 1B | 2B LE | 1B | N bytes | 2B LE | 1B |
+-----+-----+-----+-----+-----+-----+

```

File:
[SppMessage.swift](https://github.com/felipesilvax1/GalaxyBudsMacintosh/blob/master/GalaxyBudsMac/GalaxyBudsMac/Core/Protocol/SppMessage.swift)

3.3 Supported Operations

The app performs only basic companion functionality:

Operation	Direction	Description
Battery Status	Read	Left/Right/Case battery percentages
Noise Mode	Read/Write	ANC, Ambient Sound, Off
Voice Detect	Read/Write	Toggle conversation detection on/off
Voice Detect Timeout	Write	Set timeout duration (5/10/15 seconds)
Handshake	Write	Initial connection manager info

The app does not:

- Modify firmware
- Access proprietary diagnostic data
- Unlock hidden features
- Bypass any security or DRM measures
- Access any data beyond what the device voluntarily advertises over its SPP service

3.4 How the Protocol Was Documented

The protocol messages were documented through **independent analysis of publicly observable Bluetooth traffic** - specifically, by examining the structure of the binary stream transmitted by the earbuds over the standard RFCOMM channel. This is analogous to examining HTTP responses from a web server - the data is openly transmitted to any connected client.

This reverse engineering approach is:

- A widely accepted practice in the open-source community
- Legally protected under multiple jurisdictions (see Section 7)
- The same methodology used by dozens of approved App Store apps

4. Dependency Audit - Zero Samsung Code

4.1 No Package Managers

The project contains **no dependency manager files**:

- [No] No Package .swift (Swift Package Manager)
- [No] No Podfile (CocoaPods)
- [No] No Cartfile (Carthage)

4.2 No Samsung References in Code

A comprehensive search across all source files (.swift, .plist, .entitlements, .json, .yaml, .md) for the term "samsung" returns **zero matches in any source code file**.

The only occurrence of "Samsung" in the entire repository is in legal disclaimer text, explicitly stating non-affiliation:

> "Buds Connect is not affiliated with, endorsed by, or in any way officially connected with Samsung Electronics Co., Ltd."

4.3 Xcode Project Dependencies

The Xcode project's packageProductDependencies section is **empty** - confirmed in project.pbxproj. The only linked system frameworks are Apple's WidgetKit.framework and SwiftUI.framework (for the widget extension).

4.4 No Samsung Assets

The app contains **no Samsung logos, icons, trade dress, marketing materials, or any visual assets belonging to Samsung**. All UI elements are original designs.

5. Permissions & Sandboxing

5.1 App Entitlements

The app is **fully sandboxed** with minimal permissions:

```
com.apple.security.app-sandbox
com.apple.security.device.bluetooth
com.apple.security.application-groups
group.tech.miguellabs.galaxybuds
```

Only 3 entitlements: App Sandbox, Bluetooth access, and App Group (for sharing battery data with the widget).

5.2 Explicitly Denied Permissions

The Xcode build configuration **explicitly denies** all other resource access:

Permission	Status
Audio Input	[No] Denied
Bluetooth	[Yes] Allowed
Calendars	[No] Denied
Camera	[No] Denied
Contacts	[No] Denied
Location	[No] Denied
Incoming Network	[No] Denied
Outgoing Network	[No] Denied
Printing	[No] Denied
USB	[No] Denied

The app has no network access whatsoever. Both incoming and outgoing network connections are explicitly disabled in the build configuration.

6. App Store Precedents

Multiple apps currently available on the **Mac App Store** perform the same category of functionality - monitoring and controlling third-party Bluetooth hardware via standard system APIs, without official authorization from the hardware manufacturer:

6.1 Apps Currently on the Mac App Store

App	Developer	What It Does	Samsung Auth?
ToothFairy	C-Command Software	Connect/disconnect any Bluetooth device with one click	No
AllMyBatteries	Danylo Safronov	Battery monitoring for Sony, Bose, JBL and other 3rd-party Bluetooth devices	No
Magic Battery	OtiumApps	Battery display for any connected Bluetooth device including 3rd-party headphones	No

These apps have been **approved and remain available** on the Mac App Store. They connect to, monitor, and in some cases control third-party Bluetooth devices - the exact same category of functionality provided by Buds Connect.

6.2 Comparable Open-Source Projects

Project	Platform	Hardware Controlled	License
GalaxyBudsClient	Windows/Mac/Linux/Android	Galaxy Buds (all models)	GPL-3.0
Gadgetbridge	Android	Galaxy Buds, Xiaomi, Huawei, Garmin, Fitbit, etc.	AGPL-3.0

Both projects use the same reverse-engineering methodology and have been operating openly for years without legal challenges from hardware manufacturers.

7. Legal Framework for Interoperability

Reverse engineering for the purpose of achieving software interoperability is explicitly protected under multiple legal frameworks:

7.1 United States

DMCA Section 1201(f) - Interoperability Exception

- **Citation:** 17 U.S.C. § 1201(f)
- **Provision:** Permits circumvention of technological protection measures when the purpose is to identify and analyze elements necessary to achieve **interoperability** of an independently created program with other programs.
- **Applicability:** Buds Connect was independently created in Swift to achieve interoperability between the user's existing hardware and macOS - a platform the hardware manufacturer does not officially support.

Sega Enterprises Ltd. v. Accolade, Inc. (1992)

- **Court:** U.S. Court of Appeals, Ninth Circuit - 977 F.2d 1510
- **Holding:** Reverse engineering (disassembly) of software is **fair use** when it is the only way to access unprotected functional elements needed for compatibility/interoperability.
- **Relevance:** The Galaxy Buds SPP message format is not publicly documented by Samsung, making reverse engineering the only means to achieve macOS interoperability.

Sony Computer Entertainment v. Connectix Corp. (2000)

- **Court:** U.S. Court of Appeals, Ninth Circuit - 203 F.3d 596
- **Holding:** Even extensive intermediate copying during reverse engineering is permissible fair use, provided the final product does not contain the original copyrighted code and the purpose is interoperability.
- **Relevance:** Buds Connect contains zero Samsung code - the final product is 100% independently written Swift.

7.2 European Union

EU Software Directive (Directive 2009/24/EC), Article 6

- Decompilation is permitted when **indispensable** to obtain interoperability information for an independently created program.
- **Contractual provisions that attempt to prohibit these acts are null and void.**

7.3 Summary

The reverse engineering performed to document the Galaxy Buds binary message format:

- Was conducted on **publicly observable Bluetooth traffic** (not encrypted/DRM-protected content)
- Served the sole purpose of **interoperability** with macOS
- Resulted in a **100% independently written** application containing zero Samsung code
- Falls squarely within the protections of DMCA §1201(f), EU Directive 2009/24/EC, and established case law

8. Bluetooth SIG Standards Compliance

8.1 RFCOMM/SPP Is an Open Standard

- **RFComm** and **SPP (Serial Port Profile)** are defined in the Bluetooth Core Specification, published by the Bluetooth Special Interest Group (Bluetooth SIG).
- Specifications are publicly accessible at [bluetooth.com/specifications](https://www.bluetooth.com/specifications/).
- Any developer can implement applications that use RFCOMM/SPP via the operating system's Bluetooth stack.

8.2 No Separate Licensing Required

For **software-only applications** that use the operating system's existing Bluetooth stack (in this case, Apple's IOBluetooth framework):

- **No Bluetooth SIG membership** is required
- **No Bluetooth qualification process** is required
- The OS vendor (Apple) has already qualified the Bluetooth stack

Buds Connect uses Apple's IOBluetooth framework exclusively - it does not implement its own Bluetooth stack.

9. Privacy & Data Collection

9.1 No Network Activity

The app makes **zero network connections**. This is enforced at the build configuration level:

```
ENABLE_INCOMING_NETWORK_CONNECTIONS = NO  
ENABLE_OUTGOING_NETWORK_CONNECTIONS = NO
```

A search across all Swift source files for URLSession, URLRequest, HTTP, https://, or http:// returns **zero results**.

9.2 Local-Only Telemetry

The app's telemetry system (TelemetryManager.swift) uses **only Apple's MetricKit** - Apple's native framework for local diagnostic data. No data is sent to any external server. The source code explicitly states:

```
> "Evita o uso de Sentry ou envio de relatórios de uso da máquina do usuário para servidores de terceiros."  
> (Translation: "Avoids the use of Sentry or sending usage reports from the user's machine to third-party servers.")
```

9.3 Data Handling

- **Data read from earbuds:** Battery percentages, noise mode status, voice detect status
- **Data stored locally:** Battery levels cached in App Group UserDefaults (for widget display)
- **Data transmitted externally:** None
- **User accounts required:** None
- **Personal data collected:** None

10. Conclusion

Buds Connect is a straightforward companion utility that:

1. **Uses only Apple's public, documented IOBluetooth framework** - no Samsung SDK, API, or code
2. **Communicates via RFCOMM/SPP** - an open, standardized Bluetooth protocol
3. **Contains zero third-party dependencies** - only Apple system frameworks
4. **Makes zero network connections** - fully offline and privacy-respecting
5. **Is fully sandboxed** with only Bluetooth permission
6. **Was independently written** in Swift - no Samsung code was copied or embedded
7. **Is fully open-source** (GPLv3) and publicly auditable
8. **Falls within established legal protections** for reverse engineering for interoperability
9. **Is comparable to multiple apps currently approved on the Mac App Store** (ToothFairy, AllMyBatteries, Magic Battery)

We respectfully submit that Buds Connect does not infringe upon any intellectual property rights and fully complies with Guideline 5.2.1. We welcome any questions during the scheduled call and are happy to provide additional technical details or live code walkthroughs.

Prepared by: Felipe Miguel - MiguelLabs

Contact: felipemgsilva85@gmail.com

Source Code: <https://github.com/felipesilvax1/GalaxyBudsMacintosh>

Date: July 15, 2026